In [ ]:
```python
import collections
import csv
import os
import sys
import time
import marimo as mo
from ortools.sat.python import cp_model
```

Trabalho Prático: Gerador de Horário Escolar

Contexto e Enquadramento

Uma escola precisa de gerar automaticamente o horário semanal de um conjunto de turmas, respeitando a carga letiva, a disponibilidade e as salas limitadas. Este projeto implementa um sistema automatizado capaz de gerar o horário inicial e reagir de forma incremental a alterações de recursos.

Relatório Técnico do Projeto

Este documento detalha o planeamento, a arquitetura e as estratégias implementadas no desenvolvimento do sistema de geração de horários escolares.

1. Ferramenta de Modelação: OR-Tools e CP-SAT

A escolha recaiu sobre o solver **CP-SAT** da biblioteca OR-Tools. Embora tenha existido uma experimentação inicial com o solver SCIP (usado na Ficha 3), o CP-SAT revelou-se a ferramenta ideal para este problema específico de escalonamento. Problemas de alocação de horários assentam essencialmente em variáveis booleanas e em restrições lógicas e temporais, áreas onde os solvers baseados em Satisfação de Restrições (CSP) demonstram uma performance de pesquisa significativamente superior a abordagens puramente matemáticas de programação linear. O CP-SAT permite expressar as regras escolares de forma nativa e eficiente, acelerando o processo de convergência para um horário exequível.

2. Importação de Dados

A importação de dados foi implementada recorrendo unicamente à biblioteca `csv` nativa do Python (`csv.DictReader`). Esta escolha reflete uma abordagem focada na simplicidade e na redução de dependências externas desnecessárias. Como os ficheiros fornecidos contêm estruturas tabulares simples (`turmas.csv` ,

`disciplinas.csv`, `salas.csv`, `disponibilidade_excecoes.csv`), esta estrutura é facilmente manipulável e iterável aquando da geração das variáveis de decisão pelo modelo CP-SAT, não se justificando a sobrecarga de importar bibliotecas de manipulação de dados mais pesadas (como o Pandas).

3. Implementação das Restrições Obrigatórias (R1 a R8)

O modelo cria variáveis booleanas para representar se uma turma tem aula de uma determinada disciplina num dia e período. **Nota de Escala:** Em vez dos habituais 5 períodos diários, o sistema foi dimensionado para utilizar **10 períodos diários**, permitindo cobrir turnos alargados (manhã e tarde), o que oferece uma maior gama de variações, flexibilidade de horários e capacidade para acomodar mais turmas e atividades.

- **R1 e R5 (Sem sobreposição de turmas e professores):** O modelo agrupa todas as variáveis associadas a uma mesma turma ou professor num determinado período, impondo que a soma seja ≤ 1.
- **R2 (Carga letiva exata):** A soma das variáveis de uma turma para uma dada disciplina ao longo da semana é forçada a igualar rigorosamente o valor de `carga_semanal`.
- **R3 e R4 (Aulas diárias e Blocos duplos):** Para tempos simples, a soma diária é ≤ 1. Para disciplinas com `duplo_periodo=sim`, foi criada uma matriz de variáveis auxiliares booleanas que força a ocorrência de duas aulas consecutivas e válidas nos períodos p e $p-1$.
- **R6 (Indisponibilidades):** As regras baseadas em `disponibilidade_excecoes.csv` são aplicadas no momento da criação das variáveis: se o professor está indisponível num (dia, período), a variável respetiva não é sequer instanciada.
- **R7 (Limites de Salas):** As salas são mapeadas por tipo (normal ou especial). O somatório de aulas ativas num tipo de sala por período não pode exceder a quantidade física disponível na escola.
- **R8 (Input dinâmico):** Toda a geração do modelo ocorre através da iteração dinâmica dos dicionários importados, sem parâmetros rígidos (*hardcoded*).

4. Otimização (O1): Minimização de "Buracos"

A otimização principal consiste na minimização dos tempos livres isolados no horário dos professores e turmas. Para calcular um buraco, a arquitetura utiliza variáveis indicadoras de estado (`Comecou` e `Terminou`). Um período constitui um "buraco" se o dia já começou, ainda não terminou, mas não existe nenhuma aula alocada nesse período específico. A função objetivo base ($H0$) minimiza a soma total destas variáveis ao longo da semana.

A otimização principal consiste na minimização dos tempos livres isolados no horário dos professores e turmas ao longo dos 10 períodos diários. Para calcular um buraco, a arquitetura utiliza variáveis indicadoras de estado (`Comecou` e `Terminou`). Um período constitui um "buraco" se o dia já começou, ainda não terminou, mas não existe nenhuma aula alocada nesse período específico. A função objetivo base ($H0$) minimiza a soma total destas variáveis ao longo da semana.

5. Construção Incremental (R9)

Perante pequenas alterações de recursos, gerar um horário do zero resulta em longos tempos de processamento e disrupção na escola. A estratégia incremental adotada recorre à maximização da sobreposição com o horário gerado anteriormente (`horario_base.csv`). Em vez de minimizar buracos, a função objetivo passa a ser a **maximização estrita do somatório das variáveis mantidas** que já constavam no horário antigo. O solver move apenas o estritamente necessário para acomodar as novas restrições.

6. Validação Automática e Performance Incremental

Foram construídos testes de validação automática na função `executar_testes` , que audita o array final gerado pelo solver quanto a sobreposições, carga horária, espaçamento de blocos, indisponibilidades e capacidades de salas, incluindo validações lógicas adicionais (como a salvaguarda da janela de almoço nos períodos centrais). Em modo incremental, o modelo comprova uma disrupção mínima e alta eficiência, preservando ativamente o planeamento original.

In [ ]:
```python
class HorarioError(Exception):
    pass

class DadosNaoEncontradosError(HorarioError):
    pass

class SolucaoInviavelError(HorarioError):
    pass
```

In [ ]:
```python
# Adaptação para Marimo: Utilização de os.getcwd()
# pois __file__ não está disponível por defeito em todas as versões no
contexto de notebook
BASE_DIR = os.getcwd()
DADOS_DIR = os.path.join(BASE_DIR, "dados")

CONFIG = {
    "arquivos": {
        "turmas": os.path.join(DADOS_DIR, "turmas.csv"),
        "disciplinas": os.path.join(DADOS_DIR, "disciplinas.csv"),
        "salas": os.path.join(DADOS_DIR, "salas.csv"),
        "disponibilidade_excessoes": os.path.join(
            DADOS_DIR, "disponibilidade_excecoes.csv"
```

```
        ),
    },
    "solver": {"limite_tempo_segundos": 120},
}
```

In [ ]:

```
class CarregadorDados:
    @staticmethod
    def ler_csv(caminho: str) -> list[dict]:
        if not os.path.exists(caminho):
            raise DadosNaoEncontradosError(f"Ficheiro não encontrado:
{caminho}")

        try:
            with open(caminho, mode="r", encoding="utf-8") as f:
                leitor = csv.DictReader(f)
                return [linha for linha in leitor]
        except Exception as e:
            raise DadosNaoEncontradosError(f"Erro ao ler {caminho}: {e}")

    @classmethod
    def carregar_todos(cls, caminhos: dict) -> dict:
        return {
            "turmas": cls.ler_csv(caminhos["turmas"]),
            "disciplinas": cls.ler_csv(caminhos["disciplinas"]),
            "salas": cls.ler_csv(caminhos["salas"]),
            "disponibilidade_excessoes": cls.ler_csv(
                caminhos["disponibilidade_excessoes"]
            ),
        }

    @staticmethod
    def guardar_horario_csv(dados: list[dict], caminho: str):
        if not dados:
            return

        try:
            with open(caminho, mode="w", encoding="utf-8", newline="") as
f:
                cabecalho = list(dados[0].keys())
                escritor = csv.DictWriter(f, fieldnames=cabecalho)
                escritor.writeheader()
                escritor.writerows(dados)
        except Exception as e:
            raise Exception(f"Erro ao escrever o ficheiro {caminho}: {e}")
```

In [ ]:

```
class GeradorHorario:
    def __init__(
        self, dados: dict, config_solver: dict, horario_anterior: list =
None
    ):
        self.dados = dados
        self.model = cp_model.CpModel()
        self.solver = cp_model.CpSolver()
        self.horario_anterior = horario_anterior
```

```
        if "limite_tempo_segundos" in config_solver:
            self.solver.parameters.max_time_in_seconds = float(
                config_solver["limite_tempo_segundos"]
            )

        self.aulas = {}
        self.blocos_duplos = {}
        self.buracos_turmas = []
        self.buracos_prof = []
        self.dias = ["Seg", "Ter", "Qua", "Qui", "Sex"]
        self.periodos = list(range(1, 11))

        self.salas_expandidas = collections.defaultdict(list)
        for s in self.dados["salas"]:
            tipo = s["tipo"].strip().lower()
            nome_base = s["sala"].strip()
            qtd = int(s["quantidade"])
            for i in range(1, qtd + 1):
                nome_unico = f"{nome_base} {i}" if qtd > 1 else nome_base
                chave = nome_base if tipo == "especial" else "normal"
                self.salas_expandidas[chave].append(nome_unico)

        self.disc_tipo_sala = {}
        for linha in self.dados["disciplinas"]:
            nome = linha["disciplina"].strip()
            req = (
                linha["sala_especial"].strip()
                if linha["sala_especial"]
                else "normal"
            )
            self.disc_tipo_sala[nome] = req

        self.excecoes_prof = set()
        for exc in self.dados["disponibilidade_excessoes"]:
            self.excecoes_prof.add(
                (exc["professor"].strip(), exc["dia"].strip(),
int(exc["periodo"]))
            )

    def construir_modelo(self):
        self._criar_variaveis()
        self._aplicar_restricoes_basicas()
        self._aplicar_restricoes_turmas()
        self._aplicar_restricoes_professores()

        if self.horario_anterior is not None:
            self._aplicar_modo_incremental()
        else:
            self.model.Minimize(sum(self.buracos_prof) +
sum(self.buracos_turmas))

    def _aplicar_modo_incremental(self):
        aulas_antigas = set()

        for aula in self.horario_anterior:
            aulas_antigas.add(
```

```
                (
                    aula["turma"].strip(),
                    aula["disciplina"].strip(),
                    aula["dia"].strip(),
                    int(aula["periodo"]),
                )
            )

        aulas_mantidas = []
        for chave, var_aula in self.aulas.items():
            turma, disc, dia, per = chave
            if (turma, disc, dia, int(per)) in aulas_antigas:
                aulas_mantidas.append(var_aula)

        self.model.Maximize(sum(aulas_mantidas))

    def _criar_variaveis(self):
        for turma_linha in self.dados["turmas"]:
            t = turma_linha["turma"].strip()
            for disciplina_linha in self.dados["disciplinas"]:
                d = disciplina_linha["disciplina"].strip()
                prof = disciplina_linha["professor"].strip()

                for w in self.dias:
                    for p in self.periodos:
                        if (prof, w, p) in self.excecoes_prof:
                            continue

                        nome_var = f"Xsimples_{t}_{d}_{w}_{p}"
                        self.aulas[(t, d, w, p)] =
self.model.NewBoolVar(nome_var)

                if disciplina_linha["duplo_periodo"].strip().lower() ==
"sim":
                    for w in self.dias:
                        for p in range(1, 10):
                            self.blocos_duplos[(t, d, w, p)] = (

self.model.NewBoolVar(f"Xduplo_{t}_{d}_{w}_{p}")
                            )

    def _aplicar_restricoes_basicas(self):
        turmas = [t["turma"].strip() for t in self.dados["turmas"]]

        turma_dia_per = collections.defaultdict(list)
        prof_dia_per = collections.defaultdict(list)
        tipo_sala_dia_per = collections.defaultdict(list)

        for (t, d, w, p), var in self.aulas.items():
            turma_dia_per[(t, w, p)].append(var)
            tipo_s = self.disc_tipo_sala.get(d, "normal")
            tipo_sala_dia_per[(tipo_s, w, p)].append(var)
            prof = next(
                linha["professor"].strip()
                for linha in self.dados["disciplinas"]
                if linha["disciplina"].strip() == d
```

```
            )
            prof_dia_per[(prof, w, p)].append(var)

        for _, aulas_turma in turma_dia_per.items():
            self.model.Add(sum(aulas_turma) <= 1)

        for _, aulas_professor in prof_dia_per.items():
            self.model.Add(sum(aulas_professor) <= 1)

        for (tipo_s, w, p), aulas_tipo in tipo_sala_dia_per.items():
            limite_salas = len(self.salas_expandidas.get(tipo_s, []))
            self.model.Add(sum(aulas_tipo) <= limite_salas)

    def _aplicar_restricoes_turmas(self):
        turmas = [t["turma"].strip() for t in self.dados["turmas"]]

        for t in turmas:
            for disciplina_linha in self.dados["disciplinas"]:
                d = disciplina_linha["disciplina"].strip()
                carga = int(disciplina_linha["carga_semanal"])
                duplo = disciplina_linha["duplo_periodo"].strip().lower()
== "sim"
                aulas_disciplina = [
                    var
                    for (vt, vd, vw, vp), var in self.aulas.items()
                    if vt == t and vd == d
                ]
                self.model.Add(sum(aulas_disciplina) == carga)

                for w in self.dias:
                    if not duplo:
                        aulas_no_dia = [
                            var
                            for (vt, vd, vw, vp), var in self.aulas.items()
                            if vt == t and vd == d and vw == w
                        ]
                        self.model.Add(sum(aulas_no_dia) <= 1)
                    else:
                        blocos_duplos_dia = [
                            self.blocos_duplos.get((t, d, w, p), 0)
                            for p in range(1, 10)
                        ]
                        self.model.Add(sum(blocos_duplos_dia) <= 1)

                        for p in self.periodos:
                            aulas_no_periodo = [
                                var
                                for (vt, vd, vw, vp), var in
self.aulas.items()
                                if vt == t and vd == d and vw == w and vp
== p
                            ]
                            soma_aulas_periodo = sum(aulas_no_periodo)

                            bloco_atual = self.blocos_duplos.get((t, d, w,
p), 0)
```

```
                            bloco_anterior = self.blocos_duplos.get(
                                (t, d, w, p - 1), 0
                            )
                            self.model.Add(
                                soma_aulas_periodo == bloco_atual +
bloco_anterior
                            )

        for t in turmas:
            for w in self.dias:
                aulas_p5 = [
                    var
                    for (vt, vd, vw, vp), var in self.aulas.items()
                    if vt == t and vw == w and vp == 5
                ]
                aulas_p6 = [
                    var
                    for (vt, vd, vw, vp), var in self.aulas.items()
                    if vt == t and vw == w and vp == 6
                ]
                self.model.Add(sum(aulas_p5) + sum(aulas_p6) <= 0)

        for t in turmas:
            for w in self.dias:
                aulas_turma_p = {}
                for p in self.periodos:
                    aulas_no_p = [
                        var
                        for (vt, vd, vp_dia, vp), var in self.aulas.items()
                        if vt == t and vp_dia == w and vp == p
                    ]
                    aulas_turma_p[p] = sum(aulas_no_p)

                comecou_turma = {}
                terminou_turma = {}
                for p in self.periodos:
                    comecou_turma[p] = self.model.NewIntVar(
                        0, 1, f"T_Comecou_{t}_{w}_{p}"
                    )
                    terminou_turma[p] = self.model.NewIntVar(
                        0, 1, f"T_Terminou_{t}_{w}_{p}"
                    )

                for p in self.periodos:
                    self.model.Add(comecou_turma[p] >= aulas_turma_p[p])
                    if p > 1:
                        self.model.Add(
                            terminou_turma[p - 1] <= 1 - aulas_turma_p[p]
                        )
                        self.model.Add(comecou_turma[p] >= comecou_turma[p
- 1])
                        self.model.Add(terminou_turma[p] >=
terminou_turma[p - 1])

                for p in self.periodos:
                    furo_turma_p = self.model.NewIntVar(0, 1,
```

```
f"T_Furo_{t}_{w}_{p}")
                    if p == 1 or p == self.periodos[-1]:
                        self.model.Add(furo_turma_p == 0)
                    else:
                        self.model.Add(
                            furo_turma_p
                            >= comecou_turma[p - 1]
                            - terminou_turma[p]
                            - aulas_turma_p[p]
                        )

                    self.buracos_turmas.append(furo_turma_p)

    def _aplicar_restricoes_professores(self):
        professores = set(d["professor"].strip() for d in
self.dados["disciplinas"])

        for prof in professores:
            disciplinas_do_prof = {
                linha["disciplina"].strip()
                for linha in self.dados["disciplinas"]
                if linha["professor"].strip() == prof
            }

            for w in self.dias:
                aulas_prof_p = {}
                for p in self.periodos:
                    aulas_no_periodo = [
                        var
                        for (vt, vd, vw, vp), var in self.aulas.items()
                        if vw == w and vp == p and vd in
disciplinas_do_prof
                    ]
                    aulas_prof_p[p] = sum(aulas_no_periodo)

                comecou = {}
                terminou = {}
                for p in self.periodos:
                    comecou[p] = self.model.NewIntVar(
                        0, 1, f"Comecou_{prof}_{w}_{p}"
                    )
                    terminou[p] = self.model.NewIntVar(
                        0, 1, f"Terminou_{prof}_{w}_{p}"
                    )

                for p in self.periodos:
                    self.model.Add(comecou[p] >= aulas_prof_p[p])
                    if p > 1:
                        self.model.Add(terminou[p - 1] <= 1 -
aulas_prof_p[p])
                        self.model.Add(comecou[p] >= comecou[p - 1])
                        self.model.Add(terminou[p] >= terminou[p - 1])

                for p in self.periodos:
                    furo_no_periodo = self.model.NewIntVar(
                        0, 1, f"Furo_{prof}_{w}_{p}"
```

```
                    )
                    if p == 1 or p == self.periodos[-1]:
                        self.model.Add(furo_no_periodo == 0)
                    else:
                        self.model.Add(
                            furo_no_periodo
                            >= comecou[p - 1] - terminou[p] -
aulas_prof_p[p]
                        )

                    self.buracos_prof.append(furo_no_periodo)

    def resolver(self):
        status = self.solver.Solve(self.model)
        print(f"ESTADO DO SOLVER CP-SAT: {status}")

        if status not in [cp_model.OPTIMAL, cp_model.FEASIBLE]:
            raise SolucaoInviavelError(
                "Não foi possível gerar um horário válido com as restrições
impostas."
            )

        return self._extrair_solucao()

    def _extrair_solucao(self):
        horario = []
        aulas_por_tempo_e_tipo = collections.defaultdict(list)

        for chave, variavel in self.aulas.items():
            if self.solver.Value(variavel) == 1:
                turma, disc, dia, per = chave
                tipo_s = self.disc_tipo_sala.get(disc, "normal")
                aulas_por_tempo_e_tipo[(dia, int(per), tipo_s)].append(
                    {
                        "turma": turma,
                        "dia": dia,
                        "periodo": int(per),
                        "disciplina": disc,
                    }
                )

        for (dia, per, tipo_s), lista_aulas in
aulas_por_tempo_e_tipo.items():
            salas_disponiveis = list(self.salas_expandidas.get(tipo_s, []))
            for i, aula_dict in enumerate(lista_aulas):
                sala_atribuida = salas_disponiveis[i %
len(salas_disponiveis)]
                aula_dict["sala"] = sala_atribuida
                horario.append(aula_dict)

        return {"status": "sucesso", "horario": horario}
```

In [ ]:

```
def executar_testes(horario: list, dados: dict):
    print("A iniciar testes de validação do horário...")

    prof_por_disc = {
```

```
        d["disciplina"].strip(): d["professor"].strip()
        for d in dados["disciplinas"]
    }
    carga_por_disc = {
        d["disciplina"].strip(): int(d["carga_semanal"])
        for d in dados["disciplinas"]
    }
    duplo_por_disc = {
        d["disciplina"].strip(): d["duplo_periodo"].strip().lower() ==
"sim"
        for d in dados["disciplinas"]
    }
    sala_req_por_disc = {
        d["disciplina"].strip(): (
            d["sala_especial"].strip() if d["sala_especial"].strip() else
"normal"
        )
        for d in dados["disciplinas"]
    }

    excecoes_prof = set()
    for exc in dados["disponibilidade_excessoes"]:
        excecoes_prof.add(
            (exc["professor"].strip(), exc["dia"].strip(),
int(exc["periodo"]))
        )

    cap_salas = collections.defaultdict(int)
    for s in dados["salas"]:
        tipo = s["tipo"].strip().lower()
        chave = s["sala"].strip() if tipo == "especial" else "normal"
        cap_salas[chave] += int(s["quantidade"])

    turmas_existentes = set(t["turma"].strip() for t in dados["turmas"])
    erros = []

    aulas_turma = collections.defaultdict(list)
    aulas_prof = collections.defaultdict(list)
    aulas_sala = collections.defaultdict(int)
    contagem_carga = collections.defaultdict(int)

    for aula in horario:
        t = aula["turma"]
        d = aula["dia"]
        p = int(aula["periodo"])
        disc = aula["disciplina"]
        prof = prof_por_disc.get(disc, "Desconhecido")
        req_sala = sala_req_por_disc.get(disc, "normal")

        aulas_turma[(t, d, p)].append(disc)
        aulas_prof[(prof, d, p)].append(t)
        aulas_sala[(req_sala, d, p)] += 1
        contagem_carga[(t, disc)] += 1

        if (prof, d, p) in excecoes_prof:
            erros.append(
```

```
                f"R6 FALHOU: {prof} a lecionar a {t} (Dia {d}, Período {p})
mas devia estar indisponível."
            )

        if p in [5, 6]:
            erros.append(
                f"EXTRA FALHOU: Aula alocada na hora de almoço (Período
{p}) para a turma {t}."
            )

    for chave, aulas in aulas_turma.items():
        if len(aulas) > 1:
            erros.append(
                f"R1 FALHOU: Turma {chave[0]} com sobreposição ({chave[1]}
P{chave[2]}): {aulas}"
            )

    for chave, turmas in aulas_prof.items():
        if len(turmas) > 1:
            erros.append(
                f"R5 FALHOU: Professor {chave[0]} com sobreposição
({chave[1]} P{chave[2]}): turmas {turmas}"
            )

    for chave, contagem in aulas_sala.items():
        tipo_req, d, p = chave
        if contagem > cap_salas.get(tipo_req, 0):
            erros.append(
                f"R7 FALHOU: Limite de salas '{tipo_req}' excedido ({d}
P{p}). Usadas: {contagem}, Máx: {cap_salas.get(tipo_req, 0)}"
            )

    for t in turmas_existentes:
        for disc in prof_por_disc.keys():
            carga_real = contagem_carga.get((t, disc), 0)
            carga_esperada = carga_por_disc[disc]

            if carga_real != carga_esperada:
                erros.append(
                    f"R2 FALHOU: Turma {t} tem {carga_real} aulas de
{disc}, mas devia ter {carga_esperada}."
                )

            dias_com_aula = collections.defaultdict(list)
            for aula in horario:
                if aula["turma"] == t and aula["disciplina"] == disc:
                    dias_com_aula[aula["dia"]].append(int(aula["periodo"]))

            duplo = duplo_por_disc[disc]
            for dia, periodos_dia in dias_com_aula.items():
                periodos_dia.sort()
                if not duplo:
                    if len(periodos_dia) > 1:
                        erros.append(
                            f"R3 FALHOU: Turma {t} tem mais do que 1 tempo
de {disc} (não-duplo) no dia {dia}."
```

```
                        )
                else:
                    if len(periodos_dia) > 2:
                        erros.append(
                            f"R4/R3 FALHOU: Turma {t} tem mais de 2 tempos
de {disc} no dia {dia}."
                        )
                    elif len(periodos_dia) == 1:
                        erros.append(
                            f"R4 FALHOU: Turma {t} tem apenas 1 tempo
isolado de {disc} no dia {dia}."
                        )
                    elif len(periodos_dia) == 2:
                        if periodos_dia[1] - periodos_dia[0] != 1:
                            erros.append(
                                f"R4 FALHOU: Turma {t} tem bloco duplo NÃO
consecutivo de {disc} no dia {dia}: {periodos_dia}"
                            )

    if not erros:
        print(
            "[SUCESSO] Todos os testes passaram! As regras (R1-R8) e horas
de almoço (Extra) estão perfeitas.\n"
        )
    else:
        print(f"\n[FALHA] Foram encontrados {len(erros)} erro(s) no
horário:")
        for erro in erros:
            print(f"  - {erro}")
        print("\n")
```

In [ ]:

```
DIAS = ["Seg", "Ter", "Qua", "Qui", "Sex"]
PERIODOS = list(range(1, 11))

def apresentar_horarios_turmas(solucao_horario: list):
    print("\n" + "=" * 60)
    print(" " * 18 + "HORÁRIOS POR TURMA")
    print("=" * 60)

    horarios_por_turma = collections.defaultdict(dict)
    for aula in solucao_horario:
        t = aula["turma"]
        d = aula["dia"]
        p = aula["periodo"]
        disc = aula["disciplina"]
        sala = aula["sala"]

        abrev = sala[:3].lower()
        partes = sala.split()
        sala_formatada = (
            f"{abrev}{partes[-1]}" if partes[-1] and partes[-1].isdigit()
else abrev
        )

        horarios_por_turma[t][(d, p)] = f"{disc[:7]} ({sala_formatada})"
```

```
    for turma in sorted(horarios_por_turma.keys()):
        imprimir_tabela(f"Turma: {turma}", horarios_por_turma[turma])

def apresentar_horarios_professores(solucao_horario: list,
dados_disciplinas: list):
    print("\n" + "=" * 60)
    print(" " * 16 + "HORÁRIOS POR PROFESSOR")
    print("=" * 60)

    mapa_professores = {}
    for d in dados_disciplinas:
        mapa_professores[d["disciplina"].strip()] = d["professor"].strip()

    horarios_por_prof = collections.defaultdict(dict)
    for aula in solucao_horario:
        prof = mapa_professores.get(aula["disciplina"], "Desconhecido")
        d = aula["dia"]
        p = aula["periodo"]
        turma = aula["turma"]
        sala = aula["sala"]

        abrev = sala[:3].lower()
        partes = sala.split()
        sala_formatada = f"{abrev}{partes[-1]}"

        horarios_por_prof[prof][(d, p)] = f"{turma} ({sala_formatada})"

    for prof in sorted(horarios_por_prof.keys()):
        imprimir_tabela(f"Professor: {prof}", horarios_por_prof[prof])

def imprimir_tabela(titulo: str, grelha: dict):
    print(f"\n--- {titulo} ---")

    LARGURA_PER = 7
    LARGURA_DIA = 14

    linha_separadora = "-" * 92
    cabecalho = f"{'Período':^{LARGURA_PER}} | {'Seg':^{LARGURA_DIA}} |
{'Ter':^{LARGURA_DIA}} | {'Qua':^{LARGURA_DIA}} | {'Qui':^{LARGURA_DIA}} |
{'Sex':^{LARGURA_DIA}}"

    print(cabecalho)
    print(linha_separadora)

    for p in PERIODOS:
        linha = f"{str(p):^{LARGURA_PER}} | "
        blocos = []
        for dia in DIAS:
            conteudo = grelha.get((dia, p), "")
            blocos.append(f"{conteudo[:LARGURA_DIA]:<{LARGURA_DIA}}")

        linha += " | ".join(blocos)
        print(linha)

    print(linha_separadora)
```

In [ ]:

```
def main():
    try:
        print("=== SISTEMA DE GERAÇÃO DE HORÁRIOS ===")
        print("0 - Gerar Horário Base (H0)")
        print("1 - Gerar Horário Incremental (H1)")

        opcao = input("Escolha a opção: ").strip()

        if opcao not in ["0", "1"]:
            print("Opção inválida. A terminar.", file=sys.stderr)
            return  # Adaptado para Notebook (substitui sys.exit(1))

        print("\nA carregar dados iniciais...")
        dados_iniciais = CarregadorDados.carregar_todos(CONFIG["arquivos"])

        horario_anterior = None
        ficheiro_saida = "horario_base.csv"

        if opcao == "1":
            print("A carregar horário base para construção incremental...")
            try:
                horario_anterior =
CarregadorDados.ler_csv("horario_base.csv")
                ficheiro_saida = "horario_incremental.csv"
                print("A carregar alterações de recursos da pasta
'dados_v2'...")
                arquivos_v2 = {
                    chave: caminho.replace("dados", "dados_v2")
                    for chave, caminho in CONFIG["arquivos"].items()
                }
                dados_iniciais =
CarregadorDados.carregar_todos(arquivos_v2)

            except Exception as e:
                print(
                    f"Erro ao carregar o horário base (H0) necessário para
a construção incremental: {e}",
                    file=sys.stderr,
                )
                return  # Adaptado para Notebook

        print("\n--- GERAÇÃO DE HORÁRIO ---")
        inicio = time.time()
        gerador = GeradorHorario(
            dados_iniciais, CONFIG["solver"],
horario_anterior=horario_anterior
        )
        gerador.construir_modelo()
        solucao = gerador.resolver()
        tempo_total = time.time() - inicio
        print(f"Horário gerado com sucesso em {tempo_total:.2f} segundos.")

        if solucao.get("status") == "sucesso":
            horario_gerado = solucao["horario"]
            dados_disciplinas = dados_iniciais["disciplinas"]
```

```
            apresentar_horarios_turmas(horario_gerado)
            apresentar_horarios_professores(horario_gerado,
dados_disciplinas)

            print("\n" + "=" * 60)
            executar_testes(horario_gerado, dados_iniciais)
            print("=" * 60)

            CarregadorDados.guardar_horario_csv(horario_gerado,
ficheiro_saida)
            print(f"\nHorário exportado com sucesso para
'{ficheiro_saida}'.")

    except Exception as e:
        print(f"Erro inesperado: {e}", file=sys.stderr)
        return  # Adaptado para Notebook
```

Registo da interação com a LLM

In [ ]:
```
# Iniciar programa
main()
```